

Indexes, Transactions & Concurrency

Module 4 • Read plans, tune deliberately and protect shared data.

MODULE

04

90 minutes

15 min Explain
15 min Demonstrate
45 min Hands-on
15 min Review

developer_lab continues

Module 4 completes the Day 1 foundation

1

MEASURE

Read execution plans and compare evidence before changing the database.

2

OPTIMIZE

Choose a focused index that matches a real query pattern.

3

PROTECT

Use transactions and locks to keep concurrent changes correct.

Today's rule: performance changes need evidence; data changes need a transaction.

By the end, learners can explain five behaviors

01

When an index helps—and when it does not

02

How to read cost, rows, time and buffers

03

Why COMMIT and ROLLBACK define one unit of work

04

How MVCC controls what concurrent sessions see

05

Why a statement waits, deadlocks or safely continues

SUCCESS = EXPLAIN THE BEHAVIOR

An index is a workload decision—not decoration

READ BENEFIT

- Fewer pages scanned
- Faster filtering
- Useful ordering
- Faster joins

WRITE COST

- Extra disk space
- More INSERT work
- More UPDATE work
- Vacuum overhead

DECISION

- Query frequency
- Selectivity
- Table size
- Maintenance cost

Create the smallest index that clearly supports an important query.

B-tree keeps keys ordered for targeted navigation



The planner may still prefer a sequential scan when many rows are needed.

Composite index order must match the leading search pattern

```
CREATE INDEX orders_customer_created_idx  
ON app.orders (customer_id, created_at DESC);
```

STRONG MATCH

WHERE customer_id = ?
ORDER BY created_at DESC

PARTIAL MATCH

WHERE customer_id = ?
Leading column is usable.

WEAK MATCH

WHERE created_at >= ?
Leading column is absent.

Equality columns usually lead; range or ordering columns commonly follow.

Choose an index from the query—not from the table

```
SELECT order_id, status, created_at
FROM app.orders
WHERE customer_id = 42
  AND status = 'PAID'
ORDER BY created_at DESC
LIMIT 20;
```

1

Filter equality

2

Preserve useful order

3

Return few rows

```
CREATE INDEX orders_paid_customer_recent_idx
ON app.orders (customer_id, created_at DESC)
WHERE status = 'PAID';
```

EXPLAIN predicts; EXPLAIN ANALYZE executes

EXPLAIN

Shows the chosen plan and estimated work.

Safe for a statement you do not want to execute.

EXPLAIN (ANALYZE, BUFFERS)

Runs the statement and records actual rows, time and buffer activity.

Use care with writes.

For UPDATE or DELETE, test inside BEGIN ... ROLLBACK when using ANALYZE.

Read a plan from the deepest operation upward

```
Index Scan using products_price_idx on products
(cost=0.29..8.45 rows=8 width=48)
(actual time=0.021..0.029 rows=9 loops=1)
Index Cond: (price >= 100 AND price <= 110)
Buffers: shared hit=5
```

SCAN

What access method?

ROWS

Estimate versus actual?

BUFFERS

How much data touched?

Large estimate errors often point to stale statistics or skewed data.

Scan type tells you how PostgreSQL reached the rows

SEQUENTIAL SCAN

Reads table pages directly.

Often correct for small tables or queries returning many rows.

INDEX SCAN

Walks the index and fetches matching table rows.

Best for selective access.

BITMAP SCAN

Builds a page map, then reads grouped table pages.

Useful for medium selectivity.

A sequential scan is not automatically a problem; evidence decides.

Demo: create enough data to make planning visible

```
INSERT INTO app.products (sku, name, price, stock_qty)
SELECT 'DEMO-' || g, 'Demo product ' || g,
      (10 + random() * 990)::numeric(10,2),
      (random() * 100)::int
FROM generate_series(1, 50000) AS g
ON CONFLICT (sku) DO NOTHING;

ANALYZE app.products;
```

NARRATE

Why table size and statistics affect planner choices.

CHECK

```
SELECT count(*)
FROM app.products;
```

Expected: ≥ 50,000 rows.

Use training data only; do not bulk-load a production database.

Demo: compare the same query before and after

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT product_id, sku, price
FROM app.products
WHERE price BETWEEN 100 AND 110;
```

```
CREATE INDEX products_price_idx
ON app.products (price);
```

```
ANALYZE app.products;
```

```
-- Run the same EXPLAIN again
```

COMPARE

Scan type and actual rows

MEASURE

Buffers and execution time

EXPLAIN

Why the choice changed—or did not

Record evidence; cached data can change timing between repeated runs.

PostgreSQL has different index methods for different operators

B-TREE

Equality, range, sort

GIN

Full text, arrays, JSONB

GiST

Ranges, geometry, nearest

BRIN

Huge naturally ordered tables

Index method must support the operator used by the query.

Transactions make multiple statements one decision

ATOMICITY

All intended changes commit—or none persist.

CONSISTENCY

Constraints and rules remain valid.

ISOLATION

Concurrent work has controlled visibility.

DURABILITY

Committed work survives failure.

Application unit of work $\neq$ one SQL statement; define the boundary explicitly.

A safe transaction has a clear beginning and ending

```
BEGIN;
```

```
UPDATE app.products  
SET stock_qty = stock_qty - 2  
WHERE sku = 'KB-100' AND stock_qty >= 2  
RETURNING stock_qty;
```

```
-- verify related work  
COMMIT;    -- or ROLLBACK;
```

BEGIN

Start the unit of work.

VERIFY

Check row count and returned values.

END

COMMIT success or ROLLBACK failure.

SAVEPOINT rolls back part of a larger transaction

```
BEGIN;  
INSERT INTO app.orders(customer_id, status) VALUES (1, 'NEW');  
SAVEPOINT before_items;  
  
-- one line item fails  
ROLLBACK TO SAVEPOINT before_items;  
  
-- correct the items, then continue  
COMMIT;
```

USE WHEN

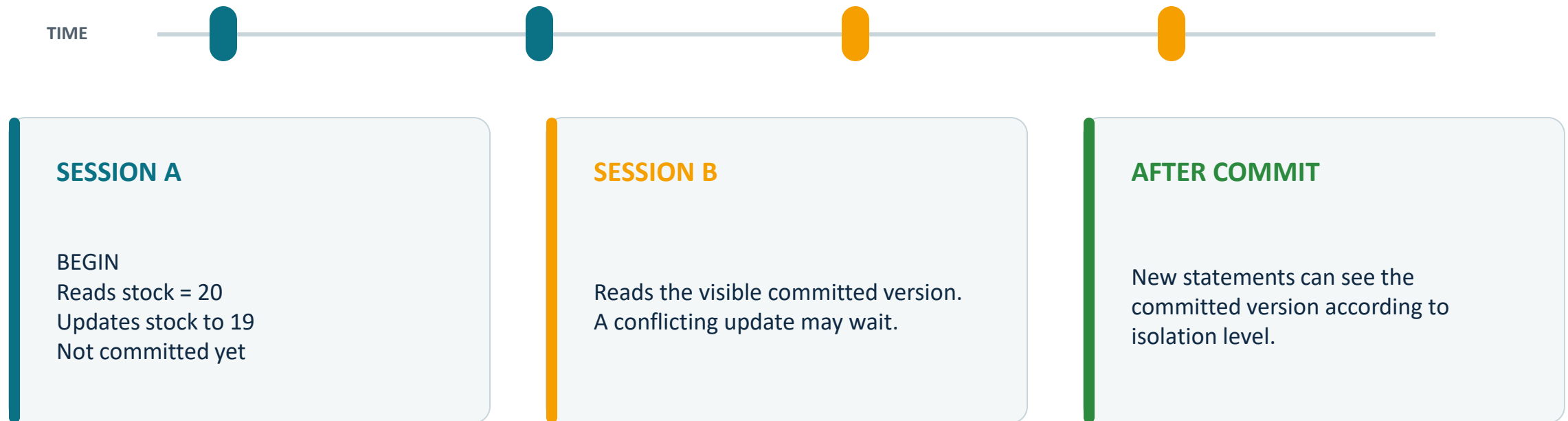
A recoverable sub-step should not cancel all earlier valid work.

DO NOT USE TO HIDE

Poor validation, unclear transaction boundaries or ignored errors.

After an unhandled error, PostgreSQL keeps the transaction aborted until rollback.

MVCC gives each statement a controlled view of rows



Readers usually do not block writers; conflicting writers can block each other.

Isolation level defines how the snapshot behaves

LEVEL	VISIBILITY	TYPICAL USE
READ COMMITTED	New snapshot per statement	Default; most application work
REPEATABLE READ	Stable transaction snapshot	Consistent multi-query analysis
SERIALIZABLE	Detects unsafe interleavings	Strongest behavior; retry may be required

Higher isolation is not “free”; design bounded retry for serialization failures.

Hands-on: observe a row lock across two sessions

SESSION A

```
BEGIN;  
UPDATE app.products  
SET stock_qty = stock_qty - 1  
WHERE sku = 'KB-100';
```

-- keep transaction open

SESSION B

```
BEGIN;  
UPDATE app.products  
SET stock_qty = stock_qty - 1  
WHERE sku = 'KB-100';
```

-- observe waiting

Then COMMIT Session A. Session B should continue; COMMIT Session B.

Observe the waiter and blocker from a third session

```
SELECT pid, username, state,  
       wait_event_type, wait_event,  
       pg_blocking_pids(pid) AS blocked_by,  
       left(query, 70) AS query  
FROM pg_stat_activity  
WHERE datname = current_database()  
ORDER BY pid;
```

FIND

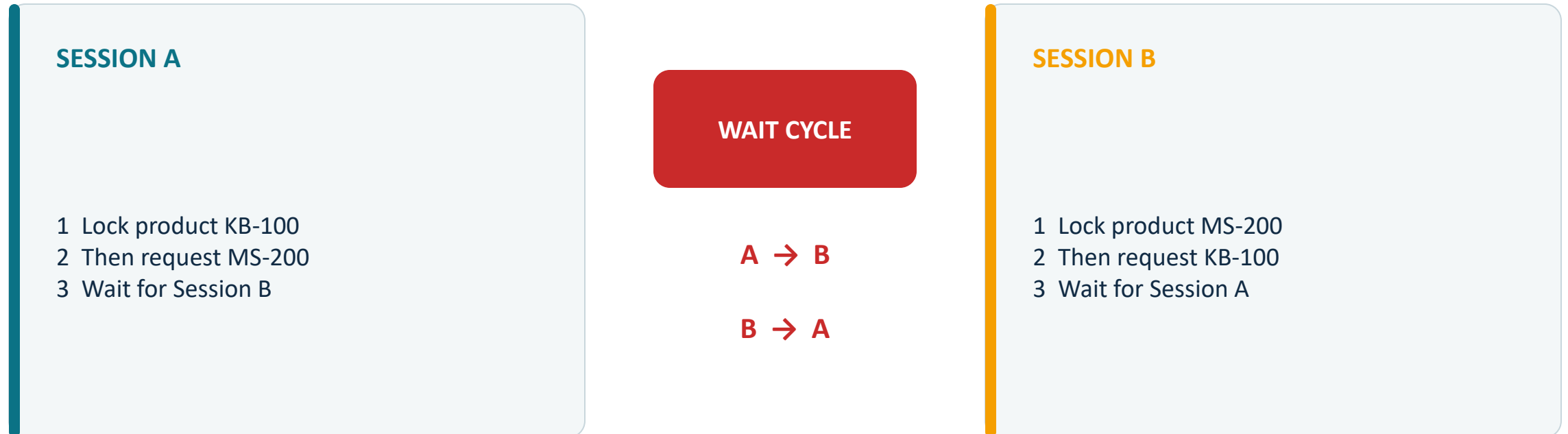
Session B has a non-empty `blocked_by` array.

EXPLAIN

Session A owns the conflicting row lock until it ends.

Success: identify the blocker PID before releasing the lock.

Deadlock: two sessions wait in opposite order



PostgreSQL aborts one transaction. Prevent cycles by locking resources in a consistent order.

Lab validation: prove performance and correctness

1

PLAN

Capture before/after scan type, rows, buffers and execution time.

2

LOCK

Show Session B waiting and identify the blocking PID.

3

RECOVER

Release the lock, commit safely and verify final stock.

Final check: `SELECT sku, stock_qty FROM app.products WHERE sku = 'KB-100';`

Troubleshoot by reading the symptom

SYMPTOM	LIKELY CAUSE	NEXT CHECK
Index still unused	Table too small, low selectivity, stale stats	ANALYZE; compare row percentage
Statement waits	Another transaction holds a lock	pg_blocking_pids(pid)
Current transaction aborted	Earlier statement failed	ROLLBACK; fix the cause
Deadlock detected	Opposite lock order	Retry safely; standardize order

Knowledge check: explain before revealing the answer

1 INDEX

Why might PostgreSQL ignore a valid index on a small table?

2 PLAN

Which two values expose a cardinality estimate problem?

3 LOCK

What releases a row lock held by an open transaction?

Answers: sequential scan can cost less; estimated vs actual rows; COMMIT or ROLLBACK.

Module 4 closes the Day 1 learning loop

MEASURE

Use EXPLAIN ANALYZE and BUFFERS to collect evidence.

DECIDE

Create focused indexes and keep transactions short.

PROTECT

Observe locks, use consistent order and retry only safe failures.

Next: Module 5 — Views and Materialized Views.